Flutter Document for what used and why

1. Dependencies used

=> I used one external dependency: the http package (version ^1.2.0).

=> I chose this package because:

* It is the official Dart/Flutter http client
* It provides simple methods: get(), post(), put(), delete().
* which map directly to Rest Api operations.
* It is lightweight and does not require complex setup.
* It is well maintained by the Dart team.

=> I also used 2 built in Dart packages:

- dart:convert -> for jsonDecode() and jsonEncode().
- dart.io -> for SocketException

2. Handling Api exceptions

=> In this app, all Api calls are wrapped in try-catch blocks inside api_service.dart. I handle 3 types of exceptions:

- SocketException (from dart:io)
  - Thrown when the device has no internet

connection of the server is unreachable.

- caught and rethrown with the message:
  "No internet connection. Please check your network."

2. HttpException (from dart:io)
   - Thrown manually when the server returns a non-2xx status code.
   - The status code is included in the message to help with debugging.

3. General Exception (catch all)
   - Catches any other unexpected errors such as: JSON parsing errors, format exceptions or timeout.
   - Every request also has a timeout (15 seconds) so the app does not wait forever for a response.

On the UI side:

- FutureBuilder checks snapshot.hasError and shows an error screen with a retry button.
- For create, edit and delete actions, a red snackbar appears at the bottom of the screen showing the error.

3. FutureBuilder widget used

=> FutureBuilder is a flutter widget that listens to a future and rebuilds its UI automatically whenever the future's state changes.

=> In this app, I declared:

late Future<List<Post>> _postsFuture;

This holds the result of _apiService.fetchPosts().

The entire body of the home screen (PostListScreen) depends on this future.

=> The FutureBuilder has 4 UI states:

1. ConnectionState.waiting
   -> shows a circularProgressIndicator spinner. and the text loading posts

2. Snapshot.hasError == true
   -> shows a red error icon, the error message, and a retry button.

3. Snapshot.hasData == true
   -> shows "No posts found." text.

The future is stored in state so that calling setState() with a new fetchPosts() triggers a full rebuild and sends a new HTTP request. This is used by the refresh button and after every create, edit or delete action.

Posts Manager

1 **sunt aut facere repellat provident occaecati excepturi optio reprehenderit**
quia et suscipit
suscipit recusandae consequuntur expedita et cum

2 **qui est esse**
est rerum tempore vitae
sequi sint nihil reprehenderit dolor beatae ea dolores neque

3 **ea molestias quasi exercitationem repellat qui ipsa sit aut**
et iusto sed quo iure
voluptatem occaecati omnis eligendi aut ad

4 **eum et est occaecati**
ullam et saepe reiciendis voluptatem adipisci
sit amet autem assumenda provident rerum culpa

5 **nesciunt quas odio**
repudiandae veniam quaerat sunt sed
alias aut fugiat sit autem sed est

+ New Post

New Post

User ID

1

Title

Content

Create Post

New Post

User ID

1

Title

Content

Create Post

New Post

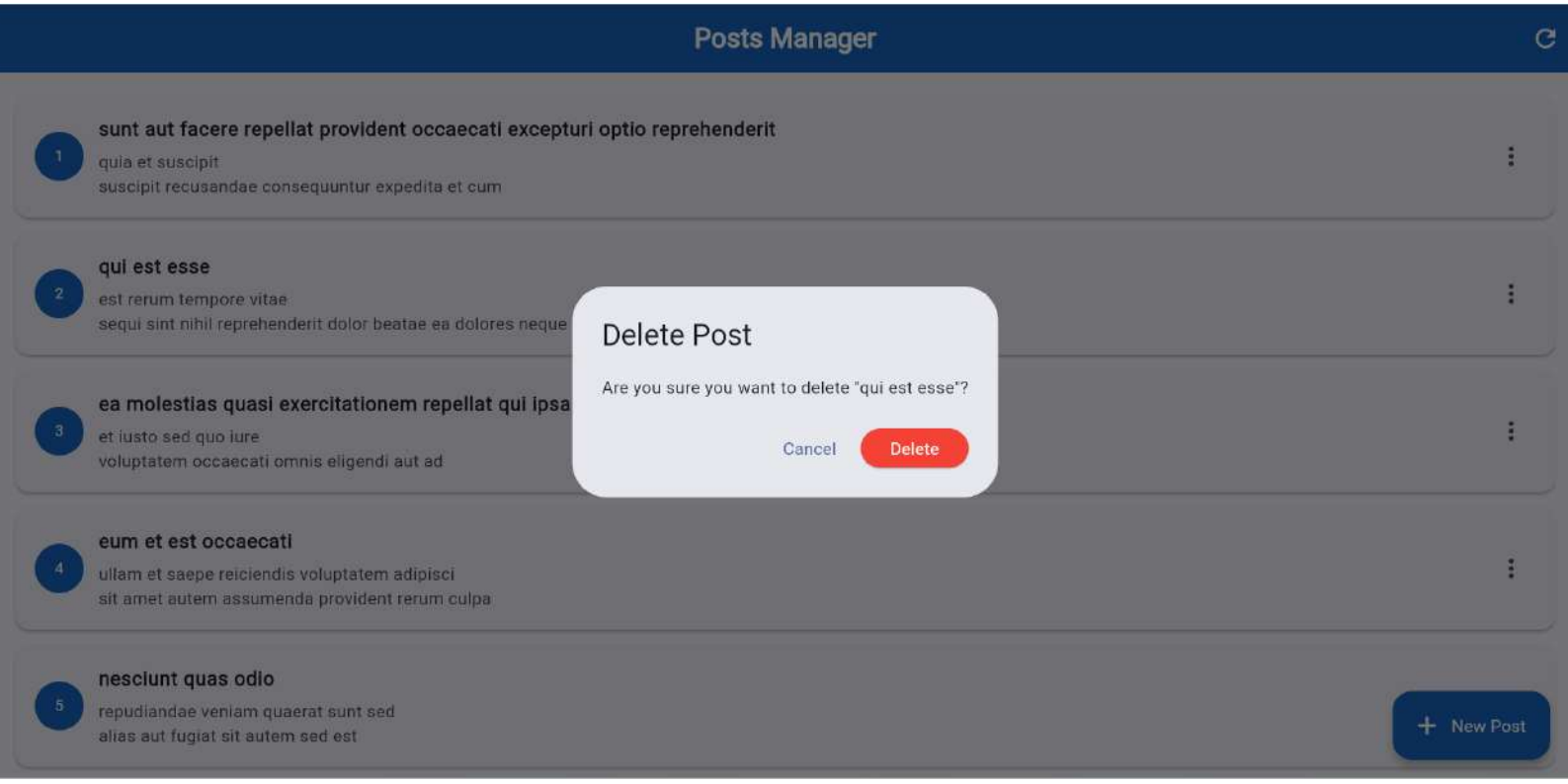
User ID

1

Title

Content

Create Post


Posts Manager
1
sunt aut facere repellat provident occaecati excepturi optio reprehenderit
quia et suscipit
suscipit recusandae consequuntur expedita et cum
2
qui est esse
est rerum tempore vitae
sequi sint nihil reprehenderit dolor beatae ea dolores neque
3
ea molestias quasi exercitationem repellat qui ipsa
et iusto sed quo iure
voluptatem occaecati omnis eligendi aut ad
4
eum et est occaecati
ullam et saepe reiciendis voluptatem adipisci
sit amet autem assumenda provident rerum culpa
5
nesciunt quas odio
repudiandae veniam quaerat sunt sed
alias aut fugiat sit autem sed est
Delete Post
Are you sure you want to delete "qui est esse"?
Cancel
Delete
New Post